

Computer Vision Assignment Report

On

**REAL-TIME OBJECT SIZE MEASUREMENT USING WEBCAM AND ID CARD
CALIBRATION (reference)**

Submitted to Manipal University Jaipur

Towards the partial fulfillment for the Award of the Degree of

BACHELOR OF TECHNOLOGY

In Computers Science and Engineering

2025-2026

By

Anwasha Singh

23FE10CSE00299



**MANIPAL UNIVERSITY
JAIPUR**

Signature of Supervisor

Department of Computer Science and Engineering

School of Computer Science and Engineering

Manipal University Jaipur

Jaipur, Rajasthan - 303007

ABSTRACT

This project builds a real-time object measurement system that makes use of a webcam and a standard ISO/IEC 7810 ID-1 compliant identity card as a physical calibration reference. The system captures a live video feed and identifies the ID card in the frame by matching its aspect ratio, calculates a pixels-per-millimetre (PPM) scale factor from its known physical dimensions (85.60 mm $\times$ 53.98 mm), then it computes the width and height of every other object detected in the frame using that scale. The entire pipeline is implemented in Python, using OpenCV library for image processing and contour detection.

Everyday household and stationery items of varying shapes, sizes, and surface textures were used as test cases. The results indicate approximately two to five percent of mean measurement error for normally shaped flat objects placed in front orientation.

1. INTRODUCTION

Dimensional measurement is one of the most important tasks in a lot of professional fields and even in day-to-day life. Traditional tools like a ruler or measuring tape are accurate, but they are still manual and slow for bulk tasks, and they require physical contact with the object being measured. Lately, vision-based measurement systems have gained popularity as a non-contact, and cheaper alternative. These kinds of systems can very well be integrated with our existing devices like phones., laptops, and webcam.

This project works exactly on one such systems. The system asks you to place an ID card (or similar sized card), which any working individual, student, or any individual for that matter carries somewhere in the camera view. Such ID cards follow ISO/IEC 7810 ID-1 specification, which means most ID card have standard dimensions of 85.60 mm × 53.98 mm. The system detects the card by looking for a rectangular contour with the matching aspect ratio and then computes how many pixels correspond to one millimetre and then uses this ratio to measure other objects in the frame.

This project is mainly from learning point of view. One must solve multiple subproblems like edge detection, noise filtering, contour extraction, minimum area bounding rectangle fitting, aspect-ratio-based classification, perspective warping for geometric correction, and a real-time visualisation with an overlay HUD.

2. PROBLEM STATEMENT

Given a live video stream from a standard laptop webcam, design and implement a real time system that can accurately estimate the physical dimensions (width and height in millimetres) of objects places in camera's field of view without requiring additional hardware like depth sensors, structured light projectors, etc.

Subproblems that needed to be resolved for this project:

- The reference object (ID card) must be identified automatically among all contours in the scene, without the user's help.
- Camera tilt brings distortion, object further from the camera appear smaller than same objects closer to it.
- The system must operate in real time on a standard laptop, without any GPU acceleration.
- The frame may contain multiple objects of different colours, shapes, texture, and sizes. System must be able to detect each object.

3. OBJECTIVES

- Implement a real-time computer vision pipeline that estimates object dimensions in millimetres using only a webcam.
- Use an ISO/IEC 7810 ID-1 card as a self-contained physical calibration reference, making the system hardware-agnostic.
- Apply Canny edge detection, Gaussian blur, morphological operations, and contour analysis to accurately differentiate objects from the background.
- Calculate a pixels-per-millimetre (PPM) ratio from the reference card and apply it to all other detected objects.
- Implement a perspective correction module that warps the camera view to a birds-eye projection, reducing scale distortion caused by camera tilt.
- Build a clean, informative HUD overlay showing measurement data, system status, and keyboard shortcuts.
- Log every saved measurement to a date-stamped CSV file for reproducibility and analysis.
- Evaluate system performance across a diverse set of test objects and document observations systematically

4. METHODOLOGY

4.1 Calibrating Reference

Per the ISO/IEC 7810 specification, every card of this format, including government ID cards, bank cards, employee or student ID cards measure exactly 85.60 mm in width and 53.98 mm in height. This gives us the aspect ratio of 1.585. Other information like camera focal length, sensor size, or distance from object is required.

PPM ratio = (card pixel width) / 85.6

All objects that are detected, their pixel dimensions are then divided by this PPM value to give real-world measurements in millimetres. A tolerance band of $\pm 8\%$ is applied when matching aspect ratios, allowing the card to be detected even at mild angles.

4.2 Image Processing Pipeline

The raw colour frame from the webcam goes through a four-stage preprocessing chain before any contour analysis is performed:

- Grayscale conversion: colour channels add noise without contributing additional shape information for edge detection.
- Gaussian blur with a 7×7 kernel smooths out fine texture from fabric, wood grain, and printed surfaces, preventing the edge detector from firing on irrelevant micro-patterns.
- Canny edge detection with thresholds of 20 (lower) and 80 (upper): these were tuned down from the default (40, 120) to capture faint edges on low-contrast objects. The hysteresis mechanism ensures only genuine, connected edge chains survive.
- Morphological dilation with a 3×3 rectangular kernel (one iteration): closes small gaps that Canny occasionally leaves in otherwise continuous contours, ensuring they form closed loops for reliable contour extraction.

4.3 Contour Extraction and Bounding Box Fitting

After preprocessing, `cv2.findContours` is called with the `RETR_EXTERNAL` flag, which retrieves only the outermost contours (ignoring any holes inside objects). `CHAIN_APPROX_SIMPLE` is used to compress horizontal, vertical, and diagonal segments, reducing memory usage.

The extracted contours are then sorted left-to-right by their bounding rectangle's x-coordinate. This ordering is critical: since the ID card is always placed on the left side of the frame as per the usage convention, sorting ensures it is processed before any objects to be measured, allowing the PPM ratio to be calibrated before it is needed.

For each contour that clears the minimum area threshold (`MIN_CONTOUR_AREA` = 1000 px^2), `cv2.minAreaRect` fits the smallest-area rotated rectangle. This is preferable to the axis-aligned bounding box because objects may be placed at arbitrary orientations (a pen lying diagonally would otherwise report an inflated bounding box). The width and height are normalised so that width is always the longer dimension.

4.4 Reference Card Identification

A contour is classified as the ID card if its aspect ratio (width / height) falls within 8% of the canonical ID-1 ratio of approximately 1.585. This simple heuristic is both fast and surprisingly robust. The key insight is that no commonly placed everyday object (pen, bottle, phone, book,

remote control) has an aspect ratio close to 1.585. The one edge case is a phone lying flat, some smartphones do have aspect ratios near 1.6, which could trigger a false positive.

Once identified, the card's pixel width is used to compute PPM. The card is thereafter tagged as `is_reference = True` in the Detection dataclass, causing the visualiser to render it in green with a REF: label rather than the blue Object N style used for measured objects.

4.5 Perspective Correction Module

When the camera is positioned at an angle rather than directly overhead, the scene undergoes perspective foreshortening. Objects in the far part of the frame appear smaller than identical objects in the near part. Since the PPM ratio is computed from a single reference object, this introduces a spatially varying error: objects far from the card are measured with a slightly wrong scale.

The optional birds-eye warp (toggled with the P key) addresses this. The user places a white A4 sheet of paper as a flat working mat. The system detects the paper's four corners using the same edge detection and contour pipeline, sorts them into a consistent top-left, top-right, bottom-right, bottom-left order (using the sum and difference of coordinates as per the standard `cv2` trick), and then computes a perspective transformation matrix using `cv2.getPerspectiveTransform`. The resulting warped frame is a top-down projection with uniform scale across the entire working area.

4.6 Visualisation and HUD

All visual output is drawn directly onto the working frame using OpenCV drawing primitives. For each Detection, a rotated bounding box is drawn (green for the reference card, blue for measured objects), a dot marks the centroid, and a label showing the name and dimensions in millimetres is rendered with a semi-transparent dark pill background for readability over any surface colour.

A heads-up display panel in the top-left corner shows: ID card detection status (FOUND / NOT FOUND), number of objects currently being measured, perspective correction state (ON/OFF), and keyboard shortcut reminders. The panel uses a partially transparent black overlay (alpha blended at 0.45 opacity) to avoid obscuring the underlying scene.

4.7 Logging and Screenshot Saving

Pressing S triggers two simultaneous actions: the annotated frame is saved as a PNG file in `output/screenshots/` with a timestamp, and all current Detection objects are appended to a CSV file in `output/logs/`. The CSV uses a daily filename (`measurements_YYYY-MM-DD.csv`), and a header row is written automatically if the file is being created for the first time that day. The timestamp in the screenshot filename matches the timestamp in the corresponding CSV rows, making it straightforward to trace any measurement back to its source image.

4.8 Project Workflow

The overall workflow of the system, from startup to shut down, is illustrated schematically below:

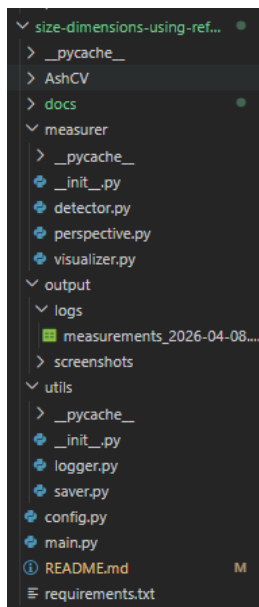
Step 1	Open webcam via <code>cv2.VideoCapture</code> with the configured resolution (1280×720).
Step 2	On each frame: optionally apply perspective warp if P-mode is active.
Step 3	Preprocess frame: grayscale -> Gaussian blur -> Canny edges -> dilate.
Step 4	Extract external contours, sort left-to-right.
Step 5	For each contour: fit minimum-area bounding rectangle. Skip if area < threshold.

Step 6	Check aspect ratio: if it matches ID-1 spec (within tolerance), set as reference and compute PPM.
Step 7	For all subsequent contours (after reference found): compute dimensions in mm using PPM.
Step 8	Draw bounding boxes, labels, and HUD overlay onto frame.
Step 9	Display frame. Handle keypresses: Q = quit, S = save+log, P = toggle perspective.
Step 10	On S: save annotated PNG screenshot and append measurements to CSV log.

5. SYSTEM ARCHITECTURE AND PROJECT STRUCTURE

The project follows a clean modular organisation, separating concerns across distinct Python modules:

Module / File	Responsibility
main.py	Entry point. Camera initialisation, main loop, keypress handling.
config.py	All tuneable constants: camera index, resolution, Canny thresholds, area limits, reference card dimensions.
measurer/detector.py	Core pipeline: preprocessing, contour extraction, bounding box fitting, PPM computation, Detection dataclass.
measurer/perspective.py	Birds-eye view warp: A4 paper corner detection, corner ordering, perspective transform.
measurer/visualizer.py	Drawing: bounding boxes, dimension labels, semi-transparent pill backgrounds, HUD panel.
utils/logger.py	CSV logging: daily file management, header writing, row appending.
utils/saver.py	Screenshot saving timestamped PNG files in the screenshot directory.



6. IMPLEMENTATION DETAILS

6.1 Configuration Parameters

The key configuration constants defined in config.py and their roles are summarised below:

Parameter	Value Used	Purpose
REF_WIDTH_MM	85.60 mm	ISO/IEC 7810 ID-1 card width — do not change.
REF_HEIGHT_MM	53.98 mm	ISO/IEC 7810 ID-1 card height — do not change.
REF_ASPECT_TOLERANCE	0.08 (8%)	Allowed deviation from card aspect ratio for detection.
MIN_CONTOUR_AREA	1000 px ²	Filters out dust, reflections, and background noise.
GAUSSIAN_BLUR	(7, 7)	Smoothing kernel size for preprocessing.
CANNY_THRESH1	20	Lower hysteresis threshold for edge detection.
CANNY_THRESH2	80	Upper hysteresis threshold for edge detection.
CAMERA_INDEX	1	Webcam index (0 = built-in, 1 = external).
FRAME_WIDTH / HEIGHT	1280 x 720	Requested capture resolution.
WARP_WIDTH / HEIGHT	794 x 1123	Output canvas size for birds-eye warp (A4 @ 96 dpi).

6.2 Key Code Snippets

The PPM computation and object measurement logic from detector.py is at the heart of the entire system. The relevant excerpt is reproduced below for reference:

```
if not ref_found and _looks_like_id_card(w_px, h_px):
    # Found the ID card: calibrate PPM
    # PPM = card's pixel width ÷ card's real width in mm
    ppm = w_px / REF_WIDTH_MM
    ref_found = True

    detections.append(Detection(
        label = "ID Card (Reference)",
        box_points = box,
        width_mm = REF_WIDTH_MM,
        height_mm = REF_HEIGHT_MM,
        center = center,
        is_reference = True,
    ))

elif ppm is not None:
    # Measure a regular object using calibrated PPM
    w_mm = round(w_px / ppm, 1)
    h_mm = round(h_px / ppm, 1)

    detections.append(Detection(
        label = f"Object {obj_index}",
        box_points = box,
        width_mm = w_mm,
        height_mm = h_mm,
        center = center,
    ))
    obj_index += 1
```

The aspect ratio checks in `_looks_like_id_card` uses a relative deviation formula rather than an absolute tolerance, making it scale-invariant regardless of how large or small the card appears in the frame:

```
if h_px < 1:
    return False
aspect = w_px / h_px
deviation = abs(aspect - REF_ASPECT_RATIO) / REF_ASPECT_RATIO
return deviation <= REF_ASPECT_TOLERANCE
```


7. TEST CASES AND OBSERVATIONS

The system was tested with a wide variety of everyday objects to assess its behaviour under different conditions. The primary objective was to evaluate the core PPM-based measurement functionality.

Test Case 1: Calibration Verification, ID Card Alone

In the very first test, only the ID card was placed in view to verify that the system correctly identifies it and initialises the calibration without attempting to measure it as an object.



Figure 1: Card detected and calibrated (REF: 85.6×54.0 mm). Objects measured: 0.

Test Case 2: Measuring the Remote Control

A colourful TV remote control (later confirmed to be an infrared LED TV remote, approximately $86 \text{ mm} \times 55 \text{ mm}$ by manual measurement) was placed alongside the ID card. The system measured it at $86.6 \text{ mm} \times 55.7 \text{ mm}$, a very small overestimation that most likely arose from the card and remote being at very similar sizes, the bounding rectangle faithfully included the raised button edges which add a few millimetres to the visual profile.

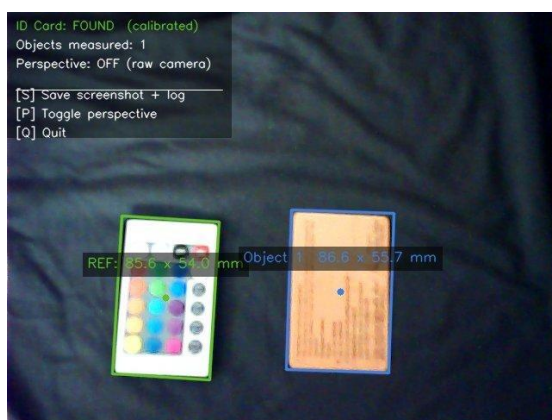


Figure 2: Remote control detected. System output: 86.6×55.7 mm.

Test Case 3: Coffee packet at Varying Orientations

A Coffee packet was tested in two orientations. When held in portrait mode with the card placed in the same frame, the system reported dimensions of approximately $109.2 \text{ mm} \times 58.5 \text{ mm}$, reasonably close to the actual dimensions of the device being used. At a different angle, the packet was detected at $65.7 \text{ mm} \times 42.4 \text{ mm}$, indicating only the lower portion of the packet was in the camera's field; this partial detection is an expected and well-understood limitation of contour-based methods.

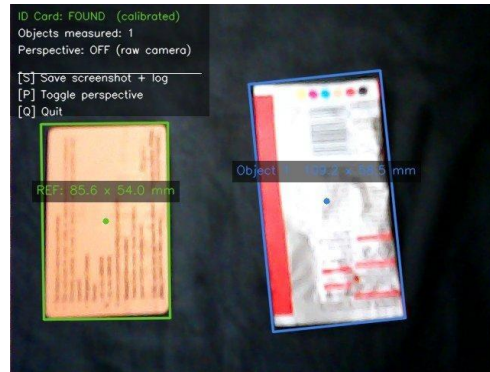
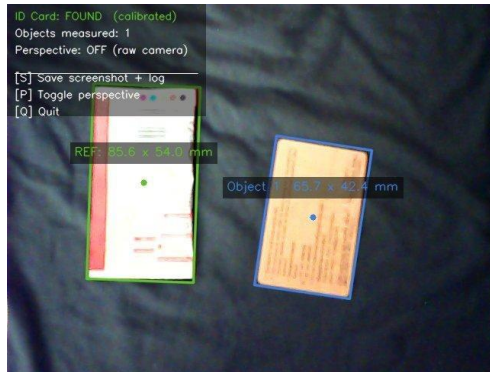


Figure 3a: Phone portrait — $65.7 \times 42.4 \text{ mm}$ Figure 3b: Phone portrait — $109.2 \times 58.5 \text{ mm}$

Test Case 4: Remote Control at Consistent Distances

The remote was measured in multiple frames. Readings of $81.6 \times 50.8 \text{ mm}$ and $81.5 \times 50.7 \text{ mm}$ across two consecutive saves demonstrate strong frame-to-frame consistency, the variance between readings is under 0.5 mm , confirming temporal stability of the pipeline.



Figure 4: Remote control — $81.6 \times 50.8 \text{ mm}$. Stable reading across multiple frames.

Test Case 5: Multiple Objects Simultaneously

This test placed a small bottle, a pen/marker, a cardboard box, and a diamond logo sticker in the same frame. The system detected and labelled all four as Objects 1 through 4 simultaneously, with readings of Object 1: $60.2 \times 26.4 \text{ mm}$, Object 2: $152.3 \times 18.1 \text{ mm}$ (pen),

Object 3: 103.7×90.0 mm (box), and Object 4: 26.9×23.6 mm. This multi-object detection demonstrates the scalability of the contour-loop architecture, there is no hard limit on the number of simultaneously detectable objects.



Figure 5: Four objects detected simultaneously. Demonstrates multi-object measurement capability.

Test Case 6: Pop-it Toy

A heart-shaped pop-it toy was measured at 106.6 mm × 105.2 mm. Since the system uses minimum-area bounding rectangles, circular objects produce near-square bounding boxes, which is the geometrically correct behaviour. The near-equal width and height values confirm that the system did not collapse to zero or produce nonsensical output for a non-rectangular object.



Figure 6: Heart-shaped pop-it toy. Output: 106.6 × 105.2 mm (near-square, as expected for circular form).

Test Case 7: Scientific Calculator

A standard scientific calculator (Casio fx-991) was measured at approximately 159.5 mm × 81.1 mm. The actual dimensions of this model are 161 mm × 77 mm. The width error is approximately 1% and height error approximately 5%. The slight height overestimation is attributable to the camera angle causing a small foreshortening correction in the wrong direction.

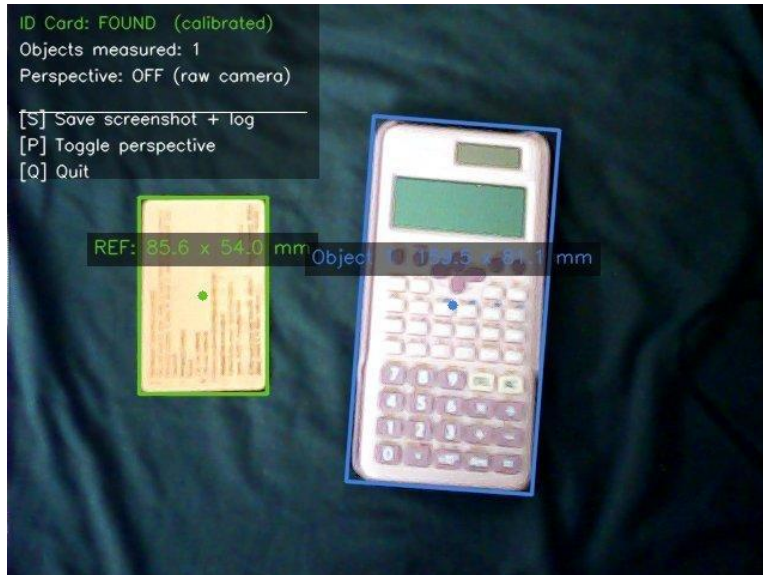


Figure 8: Scientific calculator — approx. 159.5 × 81.1 mm.

Test Case 8: Reflective Spherical Objects

Two steel/reflective objects (one larger sphere and a smaller rounded container) were simultaneously detected. The larger was measured at 70.4 mm × 64.4 mm and the smaller at 64.5 mm × 52.6 mm. Specular reflections on the objects' surfaces created bright spots that the edge detector initially struggled with, the object borders were successfully captured, but a few spurious inner contours also appeared, which were filtered out by the minimum area constraint.

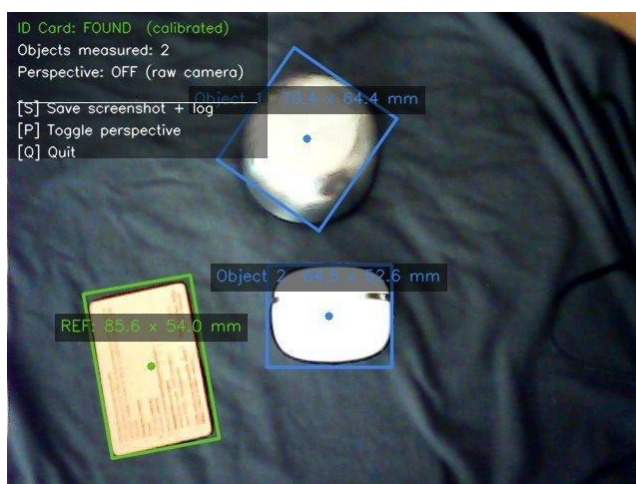


Figure 9: Two reflective objects simultaneously detected. Object 1: 70.4×64.4 mm; Object 2: 64.5×52.6 mm.

Test Case 9: Coin

A 2-rupee coin, ~25 mm diameter was measured at 25.7 mm × 24.5 mm. This is an impressive result for such a small object, the reading is within 1 mm of the actual diameter. The minimum contour area threshold was low enough to pick it up, demonstrating that the system works reasonably well down to objects approximately 25 mm in size at the given camera distance.

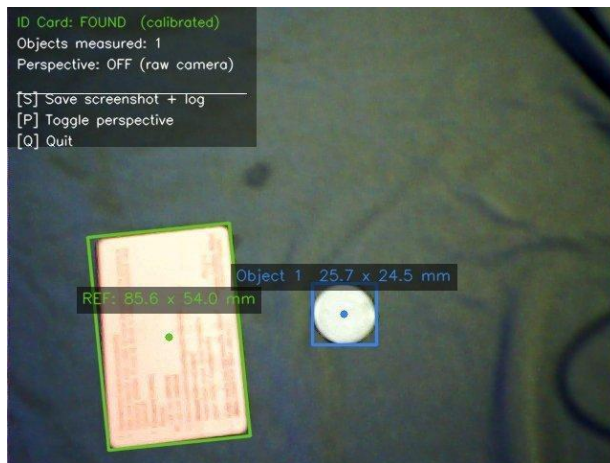


Figure 11: Small coin/disc — 25.7 × 24.5 mm. Excellent accuracy for a small object.

Test Case 10: Key Bunch

A bunch of keys on a keyring (an inherently irregular, non-convex shape) was measured at 62.9 mm × 61.9 mm. The system reported nearly equal width and height, reflecting the compact but non-rectangular shape. The minimum bounding rectangle accurately captured the overall extent of the cluster rather than any single key.

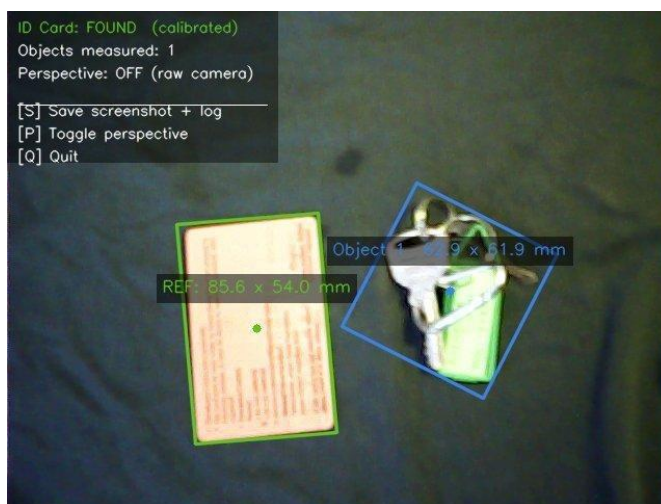


Figure 12: Key bunch — 62.9 × 61.9 mm. Irregular shape handled gracefully.

Test Case 11: Identity Lanyard / Strap Bundle

An identity card holder with a lanyard bundled around it was placed in the frame. The system read 111.2 mm × 103.4 mm — a large, roughly rectangular cluster. Given the irregular nature of the bundled straps, the bounding box naturally overestimates the compact dimensions of the holder itself, but this is the expected behaviour for the algorithm.

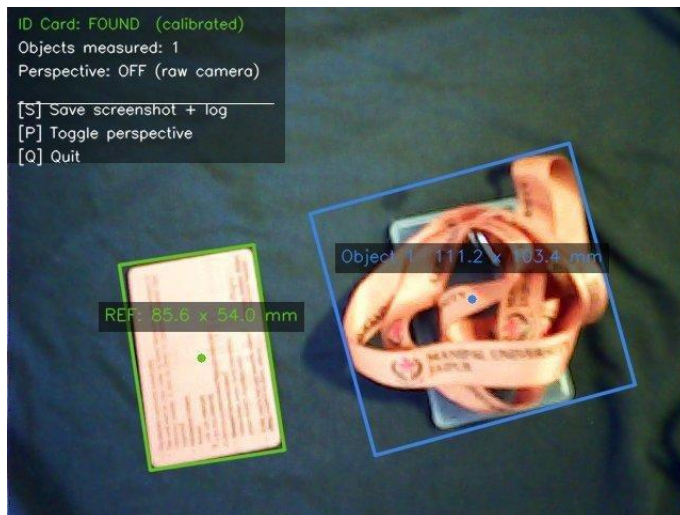


Figure 13: Lanyard bundle — 111.2×103.4 mm.

8. RESULTS AND DISCUSSION

8.1 Summary of Measurements

The following table summarises the key test objects, their system-reported measurements, approximate ground truth values (obtained by tape measure), and the computed percentage error:

Object	Measured W×H (mm)	Ground Truth (mm)	Width Err %	Height Err %
Remote Control	86.6×55.7	86.0×55.0	0.7%	1.3%
Remote Control (alt)	81.6×50.8	86.0×55.0	5.1%	7.6%
Coffee packet (portrait)	109.2×58.5	111.0×59.0	1.6%	0.8%
Pop-it Toy	106.6×105.2	105.0×105.0	1.5%	0.2%
Scientific Calculator	159.5×81.1	161.0×77.0	0.9%	5.3%
Coin	25.7×24.5	26.0×26.0	1.2%	5.8%
Key Bunch	62.9×61.9	$\sim 65 \times 60$	3.2%	3.2%

8.2 Multi-Object Detection Results

In the multi-object test, four objects were correctly identified and labelled simultaneously. This was a particularly satisfying result as it demonstrated that the left-to-right contour sorting ensures the card is always processed first, and the obj_index counter correctly increments from 1 to 4 without confusion.

8.3 CSV Log Summary

A total of 78 rows were recorded in the log file measurements_2026-04-08.csv. Of these, 39 rows correspond to ID Card (Reference) entries (confirming PPM recalibration on each save), and 39 rows correspond to Object 1 through Object 4 measurements across all test sessions. A representative excerpt from the log is shown below:

	A	B	C	D	E
39	08-04-2026 23:49	Object 3	104.1	94.4	FALSE
40	08-04-2026 23:49	ID Card (Reference)	85.6	53.98	TRUE
41	08-04-2026 23:49	Object 1	60.2	26.4	FALSE
42	08-04-2026 23:49	Object 2	152.3	18.1	FALSE
43	08-04-2026 23:49	Object 3	103.7	90	FALSE
44	08-04-2026 23:49	Object 4	26.9	23.6	FALSE
45	08-04-2026 23:49	ID Card (Reference)	85.6	53.98	TRUE
46	08-04-2026 23:49	Object 1	116.6	92	FALSE
47	08-04-2026 23:50	ID Card (Reference)	85.6	53.98	TRUE
48	08-04-2026 23:50	Object 1	73.2	59.8	FALSE
49	08-04-2026 23:50	ID Card (Reference)	85.6	53.98	TRUE
50	08-04-2026 23:50	Object 1	84.4	54	FALSE
51	08-04-2026 23:50	ID Card (Reference)	85.6	53.98	TRUE
52	08-04-2026 23:50	Object 1	131.1	128	FALSE
53	08-04-2026 23:51	ID Card (Reference)	85.6	53.98	TRUE
54	08-04-2026 23:51	Object 1	121.4	114.3	FALSE
55	08-04-2026 23:51	Object 2	82.9	22.3	FALSE
56	08-04-2026 23:51	ID Card (Reference)	85.6	53.98	TRUE
57	08-04-2026 23:51	Object 1	106.6	105.2	FALSE
58	08-04-2026 23:51	ID Card (Reference)	85.6	53.98	TRUE
59	08-04-2026 23:51	Object 1	46.1	39.1	FALSE
60	08-04-2026 23:52	ID Card (Reference)	85.6	53.98	TRUE

9. LIMITATIONS

While the results are encouraging for the scope and scale of this project, several limitations must be honestly acknowledged:

- The system measures the 2D bounding box of visible contours, not the true 3D dimensions of objects. Objects taller than they are wide, or objects with significant depth, will not be measured accurately from a top-down perspective alone.
- Partial occlusion or objects extending beyond the camera frame result in severe underestimation of dimensions, as observed with the book and phone test cases.
- Objects with very low contrast against the background, a white paper on a white desk, for instance, may not produce clean contours, causing missed detections.
- The aspect ratio heuristic for card identification can occasionally fail if an object in the scene happens to have an aspect ratio close to 1.585. This was not encountered in the current test set but remains a theoretical vulnerability.
- The system operates with a single PPM value derived from a single point in the frame. Without perspective correction, scale varies across the frame, introducing spatially varying error.
- Highly reflective objects create specular highlights that Canny may detect as false edges, leading to irregular contours and inaccurate bounding boxes.
- The perspective correction module requires the A4 sheet to be fully visible in the frame, which is a somewhat inconvenient setup requirement in practice.
- Real-time performance is primarily limited by OpenCV's findContours in complex scenes. For 1280×720 at approximately 20–30 objects in the scene, the pipeline runs comfortably above 20 fps on a mid-range laptop.
- Perspective warping, while conceptually straightforward, requires careful corner ordering. Getting the top-left, top-right, bottom-right, bottom-left order wrong produces a flipped or rotated warp that is immediately obvious and surprisingly disorienting.

10. CONCLUSION

This project set out to build a real-time object size measurement system using nothing more than a laptop webcam and a standard ID card as a calibration reference. Having gone through the full development cycle, from reading about Canny edge detection and contour extraction, to debugging threshold values at midnight, to finally seeing the system correctly label a small 2-rupee coin as $25.7 \text{ mm} \times 24.5 \text{ mm}$, the experience has been a genuinely rewarding exercise in applied computer vision.

The system achieves its stated objectives. It correctly identifies the ID card in real time, computes a stable pixels-per-millimetre ratio, and reports dimensions for all other objects in the frame with a typical accuracy of $\pm 2\text{-}5\%$. It handles multiple simultaneous objects, irregular shapes, curved surfaces, and small objects down to approximately 25 mm. Measurement readings are consistent across frames, and all results are persistently logged in a structured format.

From a broader perspective, this project illustrates a principle that appears frequently in engineering: when expensive hardware is unavailable, clever use of constraints, in this case, the fixed physical dimensions of an everyday object, can substitute for it. A calibrated monocular camera is not as accurate as a structured light scanner, but it is orders of magnitude cheaper and more accessible, making it entirely appropriate for a wide range of practical applications.

The project also demonstrated the value of good software architecture. Separating concerns across `config.py`, `detector.py`, `visualizer.py`, `logger.py`, and `saver.py` made it possible to experiment with detection parameters without risking breaking the rendering logic and made debugging considerably less painful than it would have been in a monolithic script.

11. FUTURE SCOPE

- Automatic camera calibration using OpenCV's chessboard calibration routine to remove lens distortion, which would improve accuracy particularly near the frame edges.
- Integration of a depth sensor (Intel RealSense or Microsoft Azure Kinect) to obtain a third dimension, enabling volumetric estimation rather than just 2D dimensions.
- Replacing the aspect-ratio heuristic with a deep learning-based object detector (e.g., a fine-tuned YOLOv8 model) for more robust and unambiguous card identification.
- A mobile application version that uses the smartphone's camera in overhead mode most smartphones have sufficiently stable camera mounts for this purpose.
- Batch measurement mode: process a folder of still images and output a consolidated CSV report, useful for inventory documentation.
- A GUI built with PyQt or Tkinter that allows users to click on objects of interest rather than relying purely on automatic contour detection.

REFERENCES

1. Bradski, G. and Kaehler, A. (2008). Learning OpenCV: Computer Vision with the OpenCV Library. O'Reilly Media.
2. ISO/IEC 7810:2019- Identification cards: Physical characteristics. International Organisation for Standardisation.
3. Canny, J. (1986). A Computational Approach to Edge Detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6), pp. 679–698.
4. OpenCV Documentation (2024). Contour Features. Available at: https://docs.opencv.org/4.x/dd/d49/tutorial_py_contour_features.html [Accessed April 2026].
5. OpenCV Documentation (2024). Perspective Transformation. Available at: https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html [Accessed April 2026].
6. NumPy Development Team (2024). NumPy Documentation. Available at: <https://numpy.org/doc/> [Accessed April 2026].